

Snort Rule

Snort(스노트)

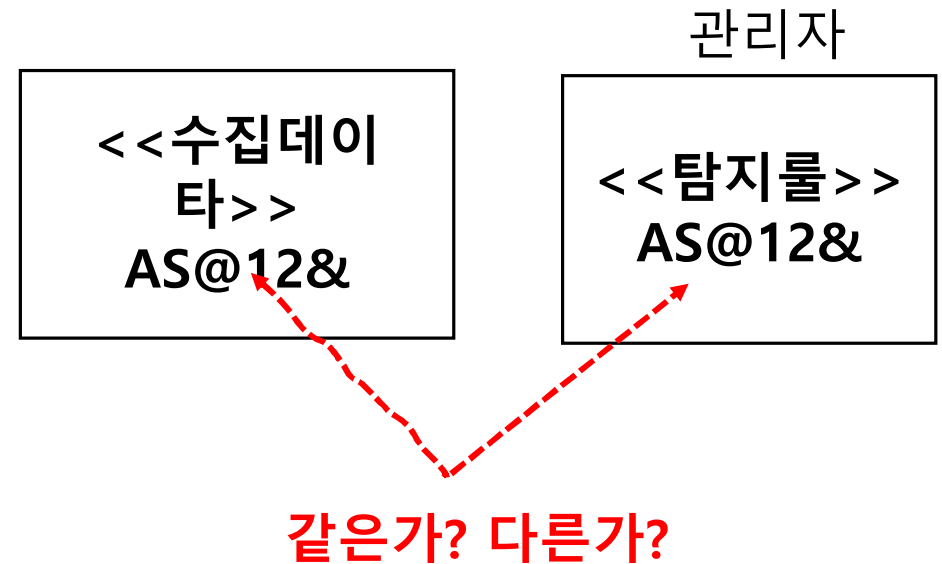
- 1998년 마틴로쉬에 의해 개발
- 오픈소스로 시그니처 기반 NIDS

- 네트워크 패킷을 수집하여 트래픽을 모니터링
- 준비된 규칙과 비교하여 침입탐지 및 경로를 발생

* 시그니처(signature) 기반이란 : 침입탐지를 문자열로 판단하는것

(패킷 데이터에서 악의적인 문자열을 탐지하여 침입여부를 결정)

- 오늘날 침입탐지시스템의 대명사로 사용



Suricata(수리카타)

- 2010년 OISF 단체에서 오픈 소스 프로젝트로 개발한 NIDS/IPS
- Snort의 단점을 개선하고 장점을 수용
 - 멀티 코어 및 멀티 스레드 지원 : 대용량 트래픽 실시간 처리(성능향상)
 - Snort Rule 완전 호환 및 대부분의 기능 지원
 - 하드웨어 벤더의 개발 지원으로 하드웨어 가속 지원
 - 스크립트 언어(Lua) 지원

Snort

- 오픈 소스로 시그니처 기반 네트워크 침입탐지 시스템 (NIDS)
- 네트워크 패킷을 수집하여 트래픽을 모니터링하고 준비된 규칙과 비교하여 침입 탐지 및 경고를 발생
- 시그니처 기반이란 침입탐지를 문자열로 판단한다는 의미
 - 악의적인 문자열을 탐색하여 침입여부를 결정



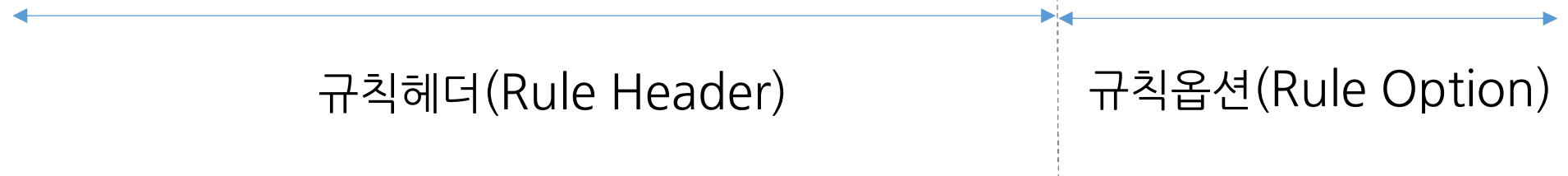
- 스니퍼 : 네트워크 패킷을 수집
- 패킷 디코더 : 수집된 패킷은 디코더로 전처리기와 탐지 엔진이 파싱 할 수 있도록 정규화
- 전처리기 : 특정 행위가 발결된 패키을 탐색 엔진에 전송
- 탐색엔진 : 해당 패킷이 스노트 규칙에 매칭 되는 지 확인
- 경고/로그 : 스노트 규칙에 매칭 된다면 콘솔 창이라 분석 도구에 경고를 출력하고 기록

Snort Rule

- Intruder Detection Rule
- NIDS나 IPS는 Pattern으로 정의된 rule을 기반으로 탐지한다.
 - Packet의 Payload를 검사하는 방식으로 공격을 탐지한다.
 - 기본적으로 signature(혹은 pattern)을 비교 혹은 검색하는 방식으로 탐지한다.
 - 정규표현식(Regular expression)으로 탐지 규칙을 규정할 수 있다.
 - DDoS 공격은 단위 시간 동안의 ‘발생량’을 기반으로 탐지한다.

Snort 시그니처 기반의 IDS 의 Detection Rule 기본 구조

```
alert tcp 192.168.10.30/32 40090 -> 192.168.10.20/32 23 (msg: "Hacker Detection";  
content:"hacker";  
nocase;  
sid:3000001;)
```



Snort Rule Header

③ 트래픽 흐름 방향						
① Action	② Protocol	Src IP	Src Port	Direction	Dst IP	Dst Port
alert	TCP	any	any	→	any	80

1 Snort Rule Header – Action

Option	Role
alert	경고를 발생하고 패킷을 로그로 저장
log	패킷을 로그로만 저장
pass	패킷을 무시하고 통과 시킴(White List 처리 시 사용)
drop	패킷을 차단 한 후 로그로 저장
reject	TCP 패킷 : 차단 및 로그 저장 후 세션을 리셋(RST 전송) UDP 패킷 : 차단 및 로그 저장 후 ICMP port unreachable 메시지를 전송
sdrop	패킷을 차단하지만 로그는 남기지 않음
activate	조건부 정밀 감시
dynamic	특정 상황에서만 활성화되도록 대기하는 가변적 액션 activate 액션에 의해 트리거 되었을 때만 작동 시작

2 Snort Rule Header – Protocol

Option	Role
tcp	TCP 프로토콜에 적용
udp	UDP 프로토콜에 적용
icmp	ICMP 프로토콜에 적용
ip	IP 프로토콜에 적용

3 Snort Rule Header – IP, Port

Option	Role
IP	any 모든 IP 주소
	1.1.1.1 특정 IP 주소
	[1.1.1.1, 2.2.2.2] 여러 IP 주소
	[1.1.1.1/24] 특정 IP 주소
PORT	Any 모든 포트 번호
	80 특정 포트 번호
	1:1024 1~1024 번 포트 범위
	80: 80 번 이상 범위
	:1024 1024번 이하 범위
	!80 80번을 뺀 나머지
→	단방향
<>	양방향

④ 규칙 옵션

- 규칙 헤더에 해당하는 패킷 중 특정 패턴(문자열)을 정의해 놓은 영역
- 옵션 종류
 - 일반옵션
 - 흐름 옵션
 - 페이로드
 - HTTP 관련 옵션 등
- 옵션들은 ‘;’(세미콜론)으로 구분

일반 옵션

- 규칙에 대한 정보를 제공하는 옵션
- 검색하는 동안 어떤 영향도 미치지 않음

msg	• 규칙이 탐지될 경우 출력되는 메시지 • 공격유형과 정보를 기록
sid	• 규칙 식별자로 모든 규칙은 반드시 식별 번호를 가짐 • 예약된 식별자 : 0~2,999,999 • Local.rules에는 3,000,000이상부터 사용
rev	• 규칙의 수정 버전을 나타냄 • 규칙이 수정 시 1 씩 증가
classtype	• 규칙을 분류하는 옵션 - 위험도와 공격 유형별로 분류 가능, 우선순위를 정할 수 있음 • 클래스 명은 classfication.config 파일에 정의되어 있음
priority	• 규칙의 우선순위 지정 • 1 ~10까지의 수 사용, 숫자가 작을수록 높은 우선순위를 가짐

Payload 옵션

content	매칭할 문자열 지정
nocase	대소문자 구별하지 않고 매칭
offset	매칭할 문자열의 위치 지정
depth	문자열의 범위 지정
distance	Content 옵션값 이후 탐색할 위치 지정
within	Content 옵션값 이후의 탐색할 범위를 지정
pcre	문자열로 표현하기 어려운 것들을 정규 표현식을 이용하여 정의 할 경우 사용

Payload 옵션

- 악성 패킷을 탐지하는 옵션

content	매칭할 문자열 지정
pcre	문자열로 표현하기 어려운 것들을 정규 표현식을 이용하여 정의 할 경우 사용

- 문자열 지정 **content: "administrator";**
- 숫자 지정 **content: "|121212|";**
- 정규표현식 지정 **pcre: "/^select/";**

“/^select/”; 검색할 문자열 중 가장 앞에 위치한 select 문자가 있는 경우 매치

예제1. Snort Rule

```
alert tcp any any -> $HOME_NET any  
(msg:"TCP NULL Scan Detected"; flags:0; sid:1000001; rev:1;)
```

[설명]

```
alert tcp any any -> $HOME_NET any  
(msg:"TCP Xmas Scan Detected"; flags:FPU; sid:1000002; rev:1;)
```

[설명]

```
alert tcp any any -> $HOME_NET any  
(msg:"TCP SYN-FIN Scan Detected"; flags:SF; sid:1000003; rev:1;)
```

[설명]

예제2. Snort Rule

```
alert tcp any any -> $HOME_NET any  
(msg:"TCP NULL Scan - Threshold Reached"; flags:0;  
threshold:type threshold track by_src, count 10, seconds 60;  
sid:1000004; rev:1;)
```

- **track by_src**: 공격자 IP(Source)를 기준 카운트
- **count 10, seconds 60**: 1분 동안 10번 이상 탐지될 경우 알람 발생

```
alert tcp any any -> $HOME_NET any  
(msg:"TCP NULL Scan - Threshold Reached"; flags:0;  
detection_filter:track by_src, count 10, seconds 60;  
sid:1000004; rev:1;)
```


Threshold

로그 발생 타입	로그 발생 기준	로그 발생 예시
threshold:type threshold , count 100, seconds 2;	조건충족 후 매번 발생한 패킷양 확인	2초내에 패킷 100개 : 로그 1개 2초내에 패킷 200개 : 로그 2개 4초내에 패킷 400개 : 로그 4개
threshold:type limit , count 100, seconds 2;	조건내 최소 한번 시간내 발생 유무 확인	2초내에 패킷 100개 : 로그 1개 2초내에 패킷 200개 : 로그 1개 4초내에 패킷 400개 : 로그 2개
threshold:type both , count 100, seconds 2;	조건 충족 후 한번 발생유무 확인	2초내에 패킷 100개 : 로그 1개 2초내에 패킷 200개 : 로그 1개 4초내에 패킷 400개 : 로그 1개

예제3. Snort Rule

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any  
(msg:"ICMP Test Incoming"; itype:8; icode:0; sid:1000001; rev:1;)
```

[설명] 외부에서 내부로 들어오는 Ping 패킷을 탐지

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 22  
(msg:"SSH Connection Attempt"; flags:S;  
threshold:type threshold, track by_src, count 5, seconds 60;  
sid:1000002; rev:1;)
```

[설명] SSH 무차별 대입 공격(Brute Force) 징후 탐지

예제 4. Snort Rule

```
alert ip any any -> $HOME_NET any  
(msg:"Excessive IP Fragmentation Detected";  
  fragbits:M;  
  threshold:type threshold, track by_src, count 100, seconds 5;  
  sid:1000009; rev:1;)
```

[설명] 5초 동안 단편화된 패킷이 100번 이상 들어오는 것을 탐지

- 단편화를 이용한 DoS 공격 유형

IP Header Rule Option

Option	Role	Example
fragbits	단편화 여부 검사 M(More fragment) D(Don't fragment), R(Reserved bit)	fragbits:M;
fragoffset	단편화된 패킷의 위치 검사	fragbits:M fragoffset:0;
ttl	ttl값 검사	ttl:=128
tos	TOS값 검사	tos:4
id	IP 헤더 ID 값 검사	id:12345;

예제 5. Snort Rule

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS 80  
(msg:"Exploit Command execution";  
flow:established,to_server;  
content:"/bin/sh";  
sid:1000013; rev:1;)
```

[설명] 세션이 이미 성립된 상태에서 외부 공격자가 서버에 명령어를 내리는 행위 탐지

flow:established: 세션이 연결된(established) 상태의 패킷만 검사

to_server: 클라이언트에서 서버로 가는 방향의 패킷 검사

TCP Header Rule Option

Option	Role	Example
Ipopts	IP헤더 옵션 값 검사	
dsize	패킷 데이터 영역 길이(byte) 검사	dsize:<1024
flow	TCP stream 전 처리로 패킷 방향 정의	flow:from_client; from_client(to_server), from_server(to_client) established
seq	순서번호	
ack	응답값	
window	TCP 헤더 윈도우값	window:55555 or window:!33333
sameip	출발지 목적지가 동일한 IP인지 조사	

예제 6. Snort Rule

외부에서 내부 서버의 80번 포트에 접속을 시도하는 패킷(SYN)이 발견되면 해당 세션의 패킷 50개를 로그 파일로 저장

① 트리거 룰 (공격 징후 포착 및 동적 룰 활성화)

```
activate tcp $EXTERNAL_NET any -> $HTTP_SERVERS 80  
(flags:S; activates:100; msg:"HTTP Connection Triggered";)
```

② 대기 룰 (활성화 시점부터 패킷 50개 기록)

```
dynamic tcp $EXTERNAL_NET any -> $HTTP_SERVERS 80  
(activated_by:100; count:50;)
```

activates:100: ID 100번으로 정의된 dynamic 룰 즉시 수행

activated_by:100: 100번 activate 룰에 의해 제어된다는 것을 명시

count:50: 활성화된 순간부터 50개의 패킷만 기록하고 다시 대기 상태로 돌아 감

예제 7. Snort Rule Header Action

① /bin/sh라는 코드가 발견되면, 그 세션에서 송수신되는 패킷 100개를 로그로 저장

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS 80
```

```
(msg:"Exploit Attempt"; content:"/bin/sh"; tag:packets, 100, session;)
```

② SSH 접속 시도가 탐지되면, 공격자 IP가 만드는 모든 트래픽을 60초 동안 기록

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 22
```

```
(msg:"SSH Brute Force"; flags:S; tag:host, 60, seconds, src;)
```


activate & tag

구분	activate + dynamic	tag
구조	2개의 룰을 유기적으로 연결해야 함	1개의 룰 내부에 옵션으로 삽입
유연성	패킷 개수(count)로만 기록 가능	시간(seconds), 바이트(bytes)로 기록
범위	특정 룰 조건에 맞는 패킷만 기록	호스트 전체나 세션 전체를 폭넓게 기록 가능
관리	룰 ID 관리 등 관리가 번거로움	유지보수가 매우 쉬움

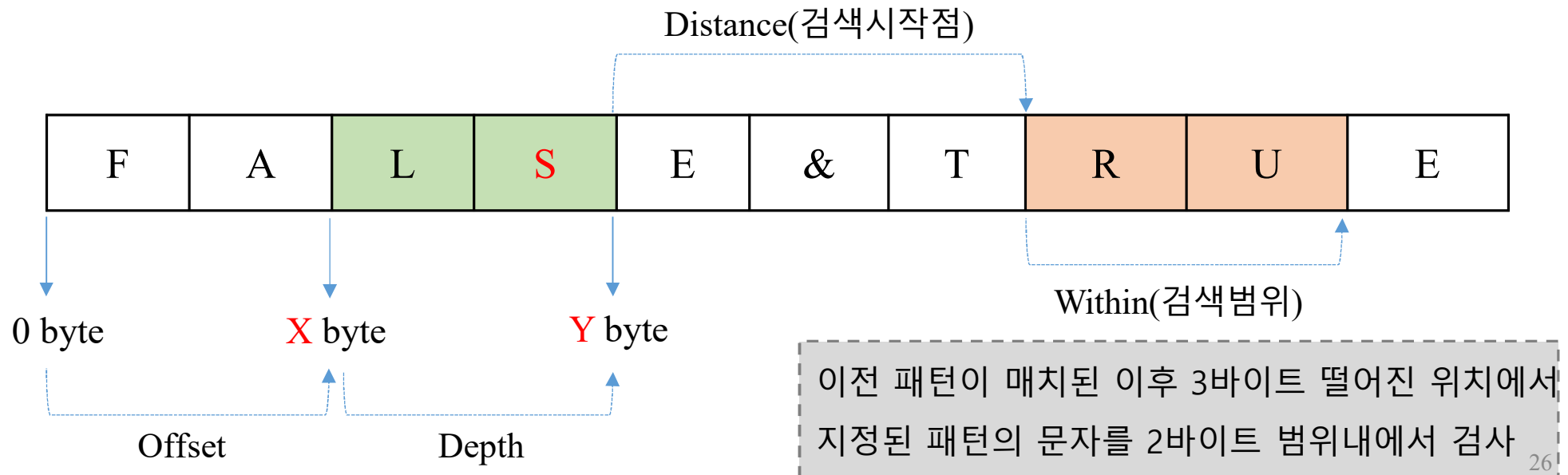
Detection Rule Example

1 Detection Rule

```
alert tcp $EXTERNAL_NET any → $HOME_NET any
```

```
(msg: "TEST"; content: "S"; offset:2; depth:2; content: "R"; distance:3; within:2;  
sid:1000001;)
```

- 3번째 byte 부터 2byte 범위에 S 패턴이 있는 지 검사, 패턴이 검사된 이 후 3byte 떨어진 위치에서 2byte 범위 내에서 R이라는 문자가 있는지 검색

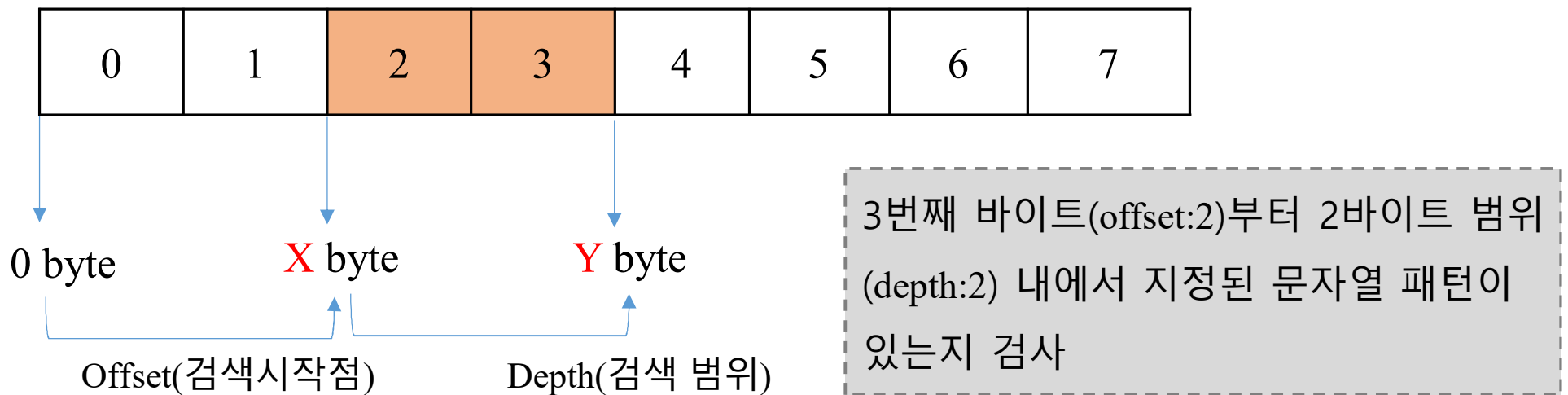


- **offset**

- content 패턴을 검사 할 시작 위치
- 첫 번째 바이트 위치가 0부터 시작

- **depth**

- offset부터 몇 바이트까지 검사할 것인지 지정

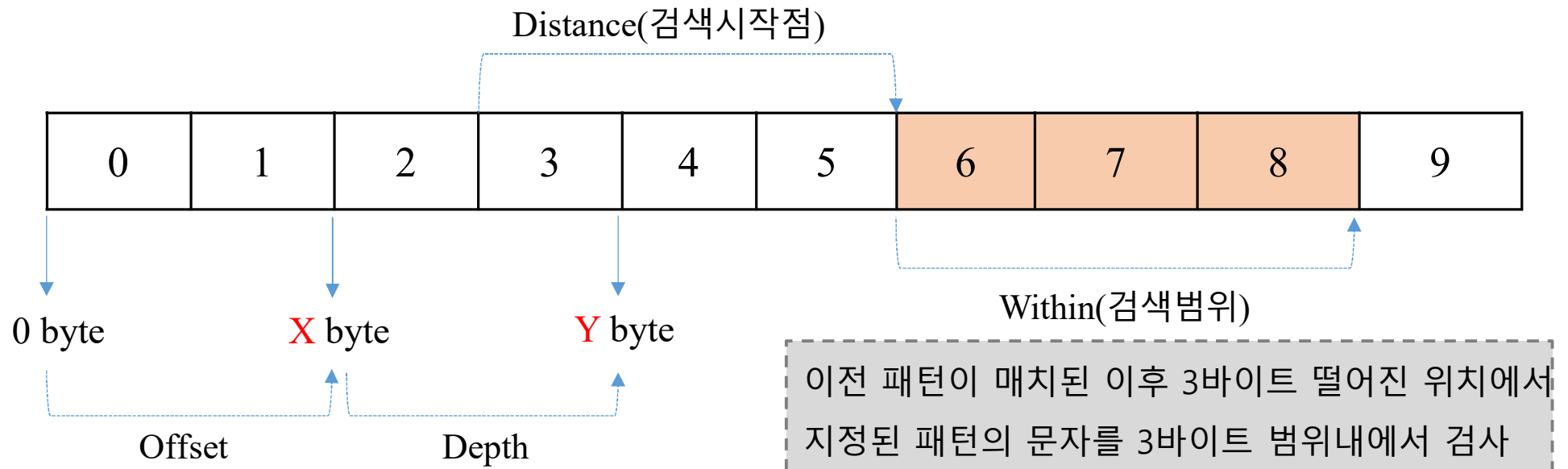


- distance

- 이전 content 패턴이 매치된 경우, 매치된 바이트로부터 몇 바이트 떨어진 위치에서 다음 content를 검사 할 것인지 지정

- within

- distance부터 몇 바이트 범위 내에서 지정된 패턴을 검사할 것인지 지정

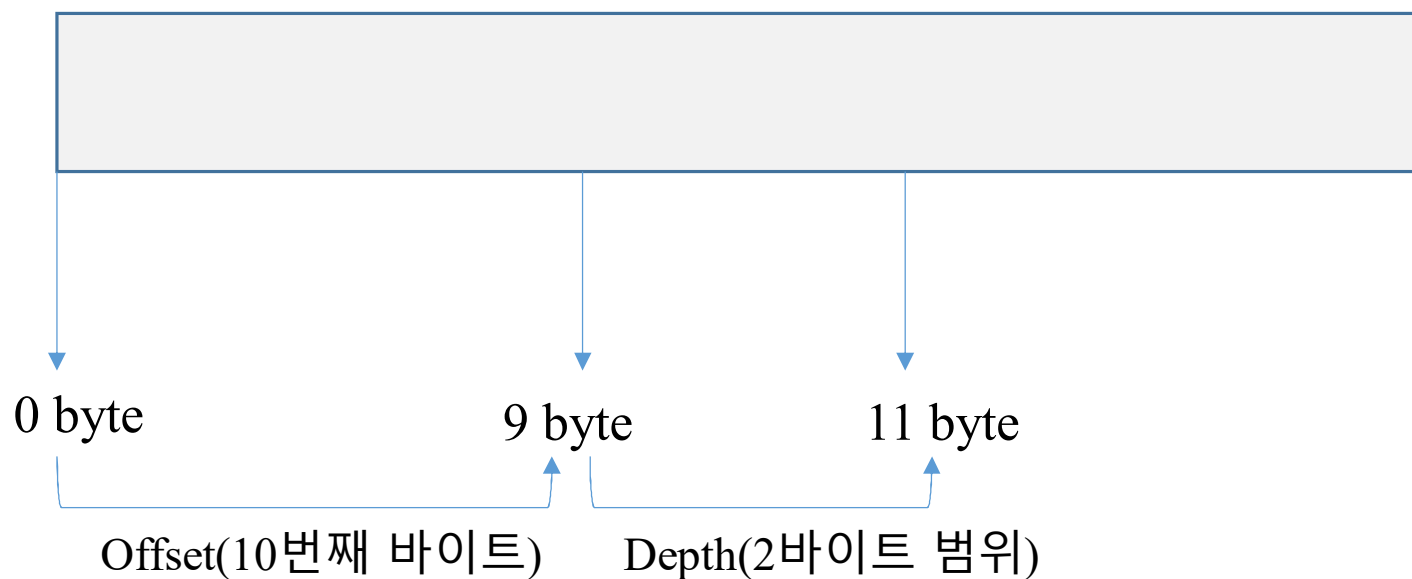


② Detection Rule

```
alert tcp $EXTERNAL_NET any → $HOME_NET any
```

```
(msg: "TEST"; content:|FFFF|; offset:9; depth:2; sid:1000001;)
```

- 10번째 byte 부터 2byte 범위에 FFFF 바이너리 패턴이 있는지 검사



3 Detection Rule

```
alert tcp any any → any 80
```

```
(msg: “ Web Scan Detected”; content: “/administrator”);
```

- 전송되는 패킷의 내용을 검사하여 “/administrator”란 문자열이 포함 된 경우 “Web Scan Detected”란 메시지로 로깅

4 Detection Rule

```
alert tcp any any → any 80 (content: “root”; nocase;)
```

- 목적지 포트가 80인 모든 TCP 패킷에 대하여 대/소문자 구분없이 페이로드에 root문자열이 포함한 경우 alert 발생

5 Detection Rule

```
alert tcp any any → any 22 (content: “login”; depth:10;)
```

- 목적지 포트가 22인 모두 TCP 패킷에 대하여 페이로드의 첫번째 byte부터 10byte 범위 내에 소문자 login 문자열이 포함한 경우 alert 발생

⑥ Detection Rule

- OpenSSL 라이브러리의 하트비트(HeartBeat) 확장 모듈의 버그로 인해 발생하는 하트블리드(heartbleed) 취약점을 이용한 공격 탐지

```
alert tcp any any <> any [443,465,563]  
(msg:"SSLv3 Malicious Heartbleed Request V2";  
content: "|18 03 00|"; depth:3;  
content: "|01|"; distance:2, within:1;  
content: "!|00|"; within:1; sid:100300;)
```

- ㉠ 첫 바이트부터 3바이트 범위 내에서 패턴 검사
- ㉡ 첫 번째 content가 매치 된 이후 2바이트 떨어진 위치에서 1바이트 내에서 지정된 패턴 검사
- ㉢ 두 번째 content가 매치된 이후 1바이트 떨어진 위치에서 지정된 패턴 검사

